1. Introduction to List, Tuple and Dictionary

Python provides built-in data structures for storing collections of values. A collection allows several values to be handled together instead of creating a separate variable for every value.

1.1 List

Simple definition: A list is an ordered and changeable collection of elements. It is written using square brackets **[ ]**.

```
numbers = [10, 20, 30, 40]
```

Characteristics:

• Elements are stored in an order.

• Indexing starts from 0.

• A list is mutable, so its elements can be changed.

• Duplicate values are allowed.

• Different types of values can be stored in one list.

• The list can grow or shrink.

Example of indexing:

Index	0	1	2	3
Value	10	20	30	40

```
numbers = [10, 20, 30]
numbers[1] = 50
# Result: [10, 50, 30]
```

The example shows mutability: the list can be changed after creation.

1.2 Tuple

Simple definition: A tuple is an ordered collection of elements that cannot normally be changed after creation. It is written using parentheses **( )**.

```
numbers = (10, 20, 30, 40)
```

Characteristics:

• Elements are ordered.

• Indexing starts from 0.

• A tuple is immutable.

• Duplicate values are allowed.

• Different types of values can be stored.

List and tuple comparison:

Feature	List	Tuple
Notation	[ ]	( )
Changeable	Yes	No
Indexing	Starts at 0	Starts at 0
Duplicates	Allowed	Allowed

1.3 Dictionary

Simple definition: A dictionary is a collection of **key-value pairs**. It is written using curly brackets **{ }**.

```
student = {"name": "Anu", "age": 18, "mark": 85}
```

In this example, **name**, **age** and **mark** are keys, while their associated values are Anu, 18 and 85.

Characteristics:

• Data is stored as key-value pairs.

• Keys are used to access values.

• Keys should be unique within a dictionary.

• A dictionary can be changed after creation.

• Dictionaries are useful when one value needs to be associated with another.

Important dictionary operations used in the frequency program:

```
frequency_dict = {}
frequency_dict[5] = 1

if 5 in frequency_dict:
    frequency_dict[5] += 1

values = list(frequency_dict.values())
```

Here, the key **5** represents an element and its value represents the number of times it has been counted.

Sorted list in non-decreasing order: [11, 12, 22, 25, 34, 64, 90]

2. Bubble Sort Program

Aim: Sort the elements of a list using a function.

```
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

    return arr


sample_list = [64, 34, 25, 12, 22, 11, 90]

sorted_list = bubble_sort(sample_list)

print(f"Sorted list in non-decreasing order: {sorted_list}")
```

2.2 Simple line-by-line explanation

1. def bubble_sort(arr): — Defines the sorting function. arr is the list received by the function.

2. n = len(arr) — Finds the number of elements in the list.

3. for i in range(n): — Starts the outer loop. It controls the passes through the list.

4. for j in range(0, n - i - 1): — Starts the inner loop and selects positions to compare. The range becomes smaller after each pass.

5. if arr[j] > arr[j + 1]: — Checks whether the left element is greater than the next element.

6. arr[j], arr[j + 1] = arr[j + 1], arr[j] — Swaps the two elements if they are in the wrong order.

7. return arr — Returns the sorted list.

8. sample_list = [...] — Creates the input list.

9. sorted_list = bubble_sort(sample_list) — Calls the function and stores the returned list.

10. print(...) — Displays the final result.

2.3 Example of the sorting process

Initial list:

```
64  34  25  12  22  11  90
```

Compare 64 and 34 → swap:

```
34  64  25  12  22  11  90
```

Compare 64 and 25 → swap:

```
34  25  64  12  22  11  90
```

Continue the comparisons. After the first complete pass:

```
34  25  12  22  11  64  90
```

The largest value, 90, has reached the last position. More passes place the remaining values correctly.

2.4 Program flow

START → Define bubble_sort() → Receive list → Find length → Start outer loop → Start inner loop → Compare adjacent elements → Is left element greater? → YES: Swap / NO: No swap → Continue comparisons → Complete all passes → Return sorted list → Print result → END

2.5 Output

3. List of Frequencies Program

Aim: Create a list of frequencies in non-increasing order using a function.

3.1 Program

```
def frequency_list(arr):
    if not arr:
        return []
```

```
        else:
            frequency_dict[item] = 1

    frequencies = list(frequency_dict.values())

    for i in range(len(frequencies)):
        for j in range(i + 1, len(frequencies)):
            if frequencies[i] < frequencies[j]:
                frequencies[i], frequencies[j] = frequencies[j], frequencies[i]

    return frequencies


sample_list = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]

frequencies = frequency_list(sample_list)

print(f"List of frequencies in non-increasing order: {frequencies}")
```

3.2 Simple line-by-line explanation

1. def frequency_list(arr): — Defines a function that receives the input list.

2. if not arr: — Checks whether the list is empty.

3. return [] — Returns an empty list if there is no input.

4. frequency_dict = {} — Creates an empty dictionary for counting.

5. for item in arr: — Takes each list element one by one.

6. if item in frequency_dict: — Checks whether the element has already been seen.

7. frequency_dict[item] += 1 — If already present, increases its frequency by 1.

8. else: — Runs when the element appears for the first time.

9. frequency_dict[item] = 1 — Stores a frequency of 1 for the first occurrence.

10. frequencies = list(frequency_dict.values()) — Takes only the dictionary values and converts them into a list.

11. for i in range(len(frequencies)): — Starts the outer sorting loop.

12. for j in range(i + 1, len(frequencies)): — Compares the current frequency with later values.

13. if frequencies[i] < frequencies[j]: — Checks whether a later frequency is larger.

14. frequencies[i], frequencies[j] = frequencies[j], frequencies[i] — Swaps them so larger frequencies move toward the beginning.

15. return frequencies — Returns the sorted frequency list.

16. sample_list = [...] — Creates the input list.

17. frequencies = frequency_list(sample_list) — Calls the function.

18. print(...) — Displays the result.

3.3 Frequency calculation

```
[3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
```

Element	3	1	4	5	9	2	6
Frequency	2	2	1	3	1	1	1

Frequency list before sorting:

```
[2, 2, 1, 3, 1, 1, 1]
```

Frequency list after non-increasing sorting:

```
[3, 2, 2, 1, 1, 1, 1]
```

3.4 Program flow

START → Define function → Receive list → Is list empty? → YES: return [] / NO: continue → Create dictionary → Read each element → Already in dictionary? → YES: increase count / NO: set count to 1 → Finish counting → Get dictionary values → Convert to list → Compare frequency values → Swap if earlier value is smaller → Complete sorting → Return list → Print result → END

3.5 Output

```
List of frequencies in non-increasing order: [3, 2, 2, 1, 1, 1, 1]
```

4. Frequency of Characters of a File

Aim: Find the frequency of characters in a given file.

4.1 Program

```
def character_frequency(filename):
    try:
        with open(filename, 'r') as file:
            text = file.read()

        frequency_dict = {}

        for char in text:
            if char in frequency_dict:
                frequency_dict[char] += 1
            else:
                frequency_dict[char] = 1

        return frequency_dict

    except FileNotFoundError:
        print(f"The file {filename} does not exist.")
        return {}


filename = 'file.txt'

f = open(filename, 'w')
f.write("hello world")
f.close()

frequencies = character_frequency(filename)

print(f"Frequencies of characters in the file: {frequencies}")
```

4.2 Simple line-by-line explanation

1. def character_frequency(filename): — Defines a function that receives the file name.

2. try: — Starts a block whose possible file-related error can be handled.

3. with open(filename, 'r') as file: — Opens the file in read mode. The with statement manages the file resource safely.

4. text = file.read() — Reads the complete file content into text.

5. frequency_dict = {} — Creates an empty dictionary for character counts.

6. for char in text: — Takes each character one by one.

7. if char in frequency_dict: — Checks whether that character has already been counted.

8. frequency_dict[char] += 1 — Increases the count when the character is already present.

9. else: — Runs for the first occurrence of a character.

10. frequency_dict[char] = 1 — Stores 1 as the first count.

11. return frequency_dict — Returns the completed character-frequency dictionary.

12. except FileNotFoundError: — Handles the situation where the requested file cannot be found.

13. print(...) — Displays a message about the missing file.

14. return {} — Returns an empty dictionary in that case.

15. filename = 'file.txt' — Stores the file name.

16. f = open(filename, 'w') — Opens the file in write mode. If needed, the file is created.

17. f.write('hello world') — Writes the text hello world into the file.

18. f.close() — Closes the file.

19. frequencies = character_frequency(filename) — Calls the function to count the characters.

20. print(...) — Displays the frequency dictionary.

4.3 Understanding the input text

```
hello world
```

The program counts every character. The blank space between the two words is also a character, so it is included in the dictionary.

For the supplied program, the resulting frequencies are:

```
{'h': 1, 'e': 1, 'l': 3, 'o': 2, ' ': 1,
```

4.4 Important file-handling terms

Term	Simple meaning
open()	Opens a file.
'r'	Read mode.
'w'	Write mode.
with open(...)	Manages the file resource safely within the block.
read()	Reads file contents.
try	Contains statements that may cause an exception.
FileNotFoundError	Error raised when the requested file is not found.
close()	Closes the opened file.

4.5 Program flow

START → Define function → Start try → Open file in read mode → Read contents → Create dictionary → Read each character → Already present? → YES: increase count / NO: set count to 1 → Return dictionary → If file missing: catch FileNotFoundError → Print error → Return empty dictionary → Create file.txt → Write 'hello world' → Close file → Call function → Print frequencies → END

4.6 Output

```
Frequencies of characters in the file:
{'h': 1, 'e': 1, 'l': 3, 'o': 2, ' ': 1, 'w': 1, 'r': 1, 'd': 1}
```

5. Quick Revision

5.1 List, Tuple and Dictionary

Data structure	Simple definition	Main characteristic	Example
List	Ordered collection	Mutable	[10, 20, 30]
Tuple	Ordered collection	Immutable	(10, 20, 30)
Dictionary	Key-value collection	Associates keys with values	{'a': 1}

5.2 Three programs — main idea

• **Bubble Sort:** Compare adjacent elements and swap when the left element is larger.

• **List of Frequencies:** Count repeated list elements using a dictionary, take the counts, and sort the counts from largest to smallest.

• **Character Frequency:** Read file text, count each character using a dictionary, and handle a missing file.

5.3 Important terms

• **Frequency:** Number of times an element or character occurs.

• **Non-decreasing:** Smallest to largest, for example 1, 2, 2, 3.

• **Non-increasing:** Largest to smallest, for example 5, 4, 4, 3, 2.

• **Index:** Position of an element; Python starts from 0.

• **Mutable:** Can be changed after creation.

• **Immutable:** Cannot normally be changed after creation.

• **Key-value pair:** A dictionary entry connecting a key with a value.

5.4 Common mistakes

• Remember that list and tuple indexing starts at 0.

• Do not try to modify a tuple element.

• In a frequency dictionary, the element is the key and the count is the value.

• For ascending Bubble Sort, swap when the left value is greater than the right value.

• For non-increasing frequency sorting, larger frequency values must move toward the beginning.

• A blank space is also a character when counting file characters.

• Use 'r' for reading and 'w' for writing.

• Use FileNotFoundError handling when a requested file may not exist.

5.5 Final summary

These programs show how Python data structures and control structures work together. Bubble Sort uses a list, a function, nested loops, comparison and swapping. The frequency-list program uses a dictionary to count repeated elements and then sorts the counts. The character-frequency program applies the same counting idea to file contents and introduces file handling and exception handling.

> **Study tip:** In a viva or exam, first state the purpose of the program, then explain the important variables, and finally describe the flow from input → processing → output.